POLITECNICO DI TORINO

01NWDBH - Mobile and Sensor Networks

Lab 04

Kubernetes



Group number:

31

Students:

Ahmad Mansour
Tala Hareb
Malak E M Alashi
Zaynab Hojeij

A.Y. 2025/26

Lab 04 - Kubernetes Report

This report answers the required questions for the Kubernetes lab in the requested Q/A format. The cluster used one control-plane node, two worker nodes, and one Client VM used as the management workstation. The main activities were service exposure, persistent storage, resource limits, autoscaling, and iperf3 network measurements. Only the most relevant screenshots are included, while the answers focus on the observed outputs and their meaning.

Q1: Briefly comment the result of the watch command.

A: The `kubectl get deployment nginx -watch` command showed the deployment moving from `4/4` ready replicas to `3/4` after one pod was deleted, and then back to `4/4`. This means Kubernetes detected that the actual state was no longer equal to the desired state of 4 replicas. The Deployment/ReplicaSet controller automatically created a replacement pod. This is the self-healing behavior of Kubernetes: the controller continuously compares the current state with the desired state and reacts when a pod disappears.

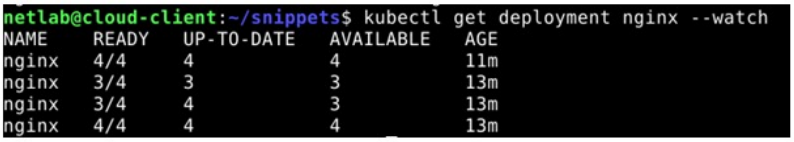


Figure 1: Q1 evidence: the watch command shows the NGINX deployment recovering back to `4/4` replicas after one pod was deleted.

Q2: Run the curl command both in the Client VM and in the control-plane one. This command works only on the control-plane/master node, why?

A: The command worked only from the control-plane node because the NGINX service was exposed as a **ClusterIP**. A ClusterIP creates an internal virtual IP reachable only inside the Kubernetes cluster network. The control-plane node belongs to this network, so it can reach the service and forward the request to the NGINX pods. The Client VM is only a management workstation outside the service network; even if it can run `kubectl`, it has no direct route to the ClusterIP range. Therefore, the Client VM curl fails because ClusterIP is internal-only, not because the deployment is broken. It is simply the wrong service type for external access from the management workstation.

Q3: Which is your <IP_MASTER_NODE>? Which output do you get with curl from the management workstation? Does it work unlike ClusterIP? Do you get similar output from the browser? Why does it work?

A: The control-plane node IP was `172.23.0.169`. The NodePort service `nginx-port` had internal ClusterIP `10.255.188.18` and port mapping `80:30100/TCP`. Running `curl http://172.23.0.169:30100` from the Client VM reached the NGINX demo application and returned the server address `10.0.2.249:80`, pod name `nginx-74b5c85c-mdv6n`, URI `/`, and a request ID. This works because NodePort exposes the service on a static port of the node IP and forwards traffic to the selected NGINX pods. Browser access to the same address gives a similar output, because both the browser request and the curl request enter the cluster through the node IP and node port.



Figure 2: Q3 evidence: browser access from the Client VM to the NodePort service at `172.23.0.169:30100`.

Q4: Briefly comment the result after deleting the MySQL pod. Which namespace, PVC, pod-local folder, events, and number of replicas are shown?

A: After creating the `trains` database and deleting the MySQL pod, Kubernetes created a new pod because the MySQL Deployment still required one replica. When reconnecting to MySQL and running `SHOW DATABASES;`, the `trains` database was still present. This proves that data was persisted through the PersistentVolume and PersistentVolumeClaim, not only inside the deleted container filesystem. The pod can be replaced, but the mounted storage is reused by the new pod.

The namespace was `default`. The PVC was `mysql-pv-claim`, bound to PV `local-pv`. The pod-local folder mapped to persistent storage was `/var/lib/mysql`; the local PV path was `/data/` on `worker-2`. The PV capacity was `1Gi`, access mode `RWO`, and reclaim policy `Retain`. The Deployment events showed `mysql-68579b78bb (1/1 replicas created)` and `Scaled up replica set mysql-68579b78bb to 1`; therefore, one MySQL replica was deployed.

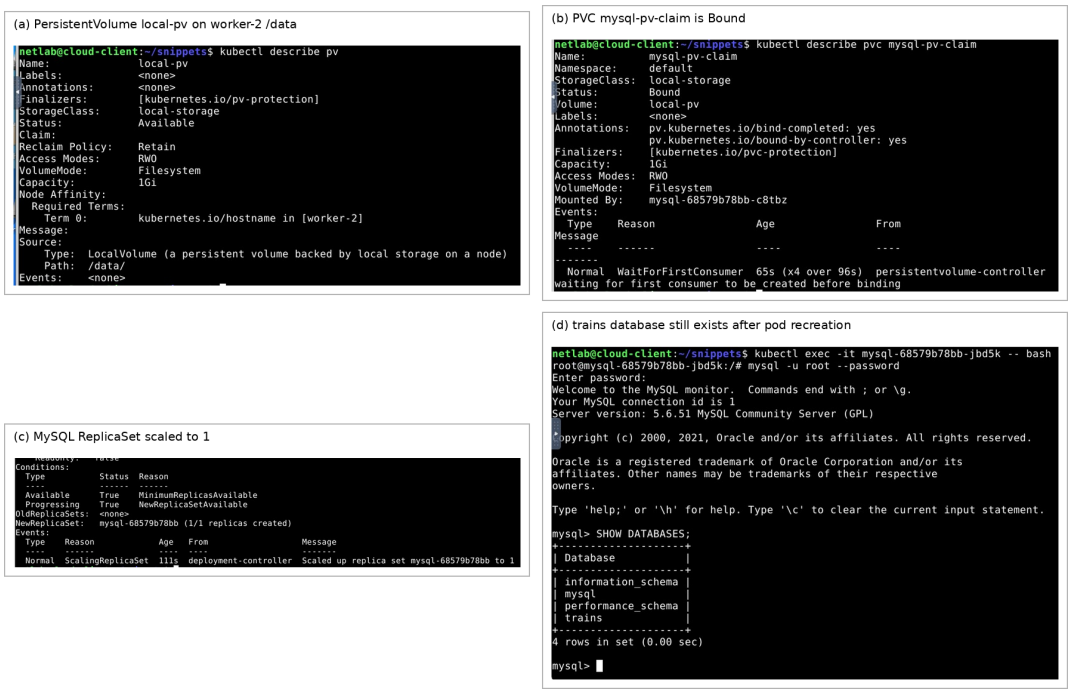


Figure 3: MySQL persistence evidence: PV/PVC binding, ReplicaSet event, and the `trains` database after pod recreation.

Q5: Identify and comment the fields stating the available resources.

A: The resource fields are `capacity` and `allocatable`, both under node `status`. `capacity` reports total node resources: in our output, `cpu: 2`, `memory: 3745844Ki`, `ephemeral-storage: 14897128Ki`, and `pods: 110`. `allocatable` reports what Kubernetes can actually assign to pods after system reservations: `cpu: 2`, `memory: 3644184Ki`, `ephemeral-storage: 13721914343`, and `pods: 110`. The scheduler mainly uses `allocatable`; it can be lower than `capacity` because some resources are reserved for the OS and Kubernetes components. For this reason, `allocatable` is the more practical value when deciding whether a new workload can fit on a node.

Q6: Compare CPU consumption before and after editing the deployment. Explain requests and limits.

A: Before editing, the stress pod consumed about `1000m` CPU, i.e., one CPU core. After setting request `150m` and limit `300m`, the pod used about `300m`; therefore, CPU was capped by the configured limit. A request is the amount reserved and used by the scheduler to place a pod. A limit is the maximum amount the container can use at runtime. CPU above the limit is throttled, while memory above the limit may terminate the container. In our case, the same stress workload stayed alive but was prevented from consuming the full CPU core.

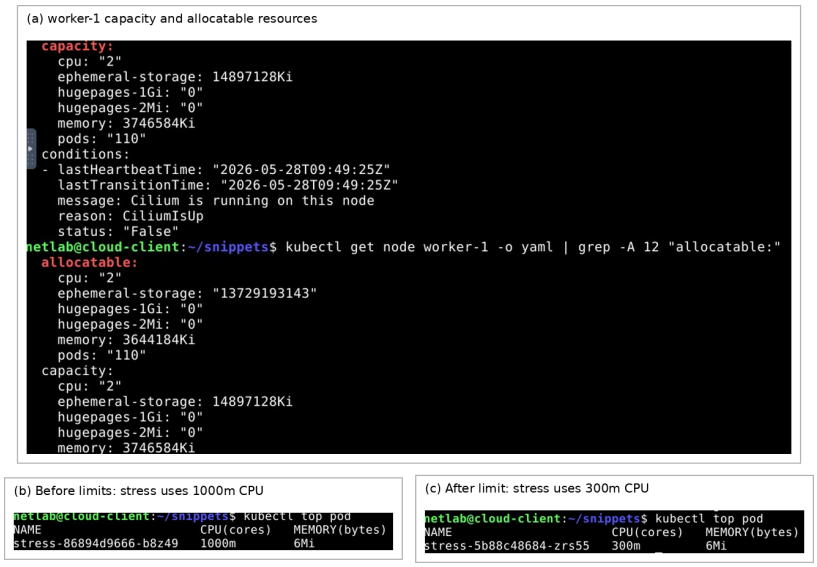


Figure 4: Resources and CPU evidence for Q5-Q6: worker-1 capacity/allocatable and stress pod CPU before/after limits.

Q7: How many replicas do you get? Is it reasonable considering the CPU target and initial high CPU? Why?

A: The HPA scaled the `php-apache` Deployment up to `7` replicas. This is shown by `kubectl get deployment php-apache`, which reported `7/7` ready replicas. This is reasonable because the target was `27%` CPU utilization and, under load, the measured CPU became close to or above the target while only one pod was initially running. Kubernetes increased replicas to distribute requests over more pods and reduce average CPU per pod. After stopping the load, CPU dropped to `0%`; after a delay, HPA scaled back down to the minimum of `1` replica. This delay is expected because the autoscaler does not immediately remove pods after every short change in CPU usage.

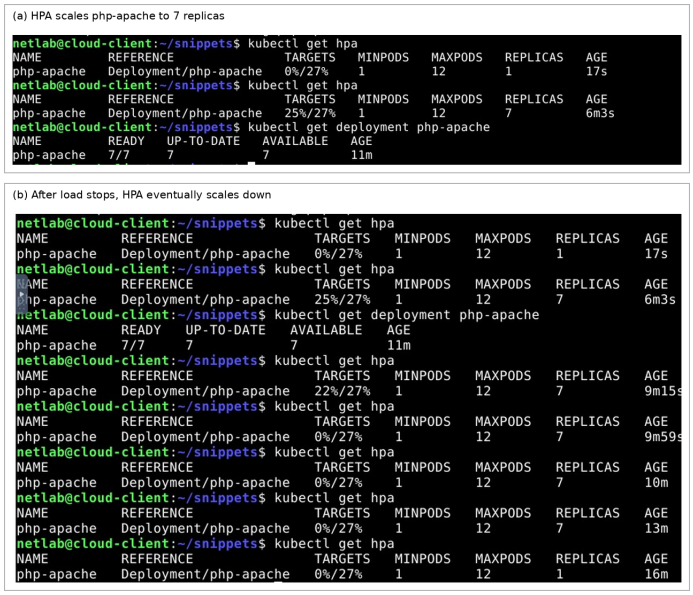


Figure 5: HPA evidence: `php-apache` scales to 7 replicas under load and later returns to the minimum.

Q8: Analyze the YAML file. List all objects and specifications, including affinity/tolerations. Is the service NodePort or ClusterIP?

A: The YAML deploys three main objects. First, the **Deployment** `iperf3-server-deployment` creates one server replica using image `networkstatic/iperf3`, argument `-s`, and port `5201`. It uses node affinity for the control-plane and tolerates the control-plane/master `NoSchedule` taint. Second, the **Service** `iperf3-server` selects `app: iperf3-server` and exposes TCP port `5201` to target port `server`. Third, the **DaemonSet** `iperf3-clients` runs one client pod on each node and also tolerates the control-plane taint. The service is **ClusterIP**, not NodePort, because the YAML does not specify `type: NodePort`; ClusterIP is the default and is enough because the clients run inside the cluster.

Q9: Describe the role of iperf3-server and iperf3-client. Why are Deployment, Service, and DaemonSet required? How many server replicas and where is this in YAML?

A: The `iperf3-server` receives benchmark traffic and measures throughput; it runs iperf3 in server mode with `-s`. The `iperf3-client` generates traffic from each node toward the server. A Deployment keeps the server pod running and defines its replica count. A Service gives clients a stable internal endpoint even if the server pod IP changes. A DaemonSet ensures one client pod runs on every node. The server uses one replica, found in the YAML under `spec.replicas: 1` of the server Deployment.

Q8-Q9 compact excerpt from iperf3.yaml (Deployment, Service, DaemonSet)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iperf3-server-deployment
spec:
  replicas: 1
  template:
    spec:
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - preference:
              matchExpressions:
              - key: node-role.kubernetes.io/control-plane
                operator: Exists
      tolerations:
      - key: node-role.kubernetes.io/master
        operator: Exists
        effect: NoSchedule
      containers:
      - name: iperf3-server
        image: networkstatic/iperf3
        args: ['-s']
        ports:
        - containerPort: 5201
          name: server
---
apiVersion: v1
kind: Service
metadata:
  name: iperf3-server
spec:
  selector:
    app: iperf3-server
  ports:
  - protocol: TCP
    port: 5201
    targetPort: server
# No type: NodePort specified -> default Service type is ClusterIP
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: iperf3-clients
spec:
  template:
    spec:
      tolerations:
      - key: node-role.kubernetes.io/master
        operator: Exists
        effect: NoSchedule
      containers:
      - name: iperf3-client
        image: networkstatic/iperf3
        command: ['/bin/sh', '-c', 'sleep infinity']
```

Figure 6: Relevant iperf3 YAML excerpt for Q8-Q9, placed beside the related answers.

Q10: Attach a screenshot with the experiment output. Briefly comment the result, especially why only one out of three repetitions shows higher throughput.

A: The iperf3 server ran on `control-plane-1`. The client on the same node, `iperf3-clients-pm5hb`, achieved the highest throughput: about `32.2 Gbits/sec` with `0` retransmissions. The worker clients were much lower: `iperf3-clients-l77h4` on `worker-2` reached about `3.55 Gbits/sec` with `70` retransmissions, while `iperf3-clients-rkkvb` on `worker-1` reached about `3.45 Gbits/sec` with `1168` retransmissions. Only one result is much higher because that client is co-located with the server, so traffic follows a local/short path. The two worker-node clients cross the Kubernetes/Cilium overlay and CrownLabs virtual network, adding overhead and retransmissions. This explains why the local measurement is much higher than the two inter-node measurements.

(a) Pod placement: server and one client on control-plane, clients on workers

```
netlab@cloud-client:~/kubernetes-iperf3$ kubectl get pods -o wide
NAME                                    READY   STATUS    RESTARTS   AGE   IP           NODE             NOMINATED NODE   READINES
S GATES
iperf3-clients-l77h4                    1/1     Running   0          73s   10.0.2.227   worker-2         <none>           <none>
iperf3-clients-pm5hb                    1/1     Running   0          73s   10.0.0.206   control-plane-1  <none>           <none>
iperf3-clients-rkkvb                    1/1     Running   0          73s   10.0.1.2     worker-1         <none>           <none>
iperf3-server-deployment-6fbfc65c5-b2rww   1/1     Running   0          73s   10.0.0.237   control-plane-1  <none>           <none>
load-generator                          1/1     Running   0          29m   10.0.1.79    worker-1         <none>           <none>
```

(b) control-plane client: 32.2 Gbits/sec, 0 retransmissions

```
netlab@cloud-client:~/kubernetes-iperf3$ kubectl exec -it iperf3-clients-pm5hb -- iperf3 -c iperf3-server
Connecting to host iperf3-server, port 5201
[  5] local 10.0.0.206 port 44858 connected to 10.255.43.75 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  4.60 GBytes  39.6 Gbits/sec    0   3.11 MBytes
[  5]   1.00-2.00   sec  4.81 GBytes  41.3 Gbits/sec    0   3.11 MBytes
[  5]   2.00-3.00   sec  4.72 GBytes  40.5 Gbits/sec    0   3.11 MBytes
[  5]   3.00-4.00   sec  3.80 GBytes  32.6 Gbits/sec    0   3.11 MBytes
[  5]   4.00-5.00   sec  3.30 GBytes  28.3 Gbits/sec    0   3.11 MBytes
[  5]   5.00-6.00   sec  4.71 GBytes  40.5 Gbits/sec    0   3.11 MBytes
[  5]   6.00-7.00   sec  4.77 GBytes  40.9 Gbits/sec    0   3.11 MBytes
[  5]   7.00-8.00   sec  2.56 GBytes  22.0 Gbits/sec    0   3.11 MBytes
[  5]   8.00-9.00   sec  2.09 GBytes  17.9 Gbits/sec    0   3.11 MBytes
[  5]   9.00-10.00  sec  2.09 GBytes  18.0 Gbits/sec    0   3.11 MBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  37.4 GBytes  32.2 Gbits/sec    0             sender
[  5]   0.00-10.00  sec  37.4 GBytes  32.2 Gbits/sec                  receiver

iperf Done.
```

(c) worker-2 client: 3.55 Gbits/sec, 70 retransmissions

```
netlab@cloud-client:~/kubernetes-iperf3$ kubectl exec -it iperf3-clients-l77h4 -- iperf3 -c iperf3-server
Connecting to host iperf3-server, port 5201
[  5] local 10.0.2.227 port 48184 connected to 10.255.43.75 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec   296 MBytes  2.48 Gbits/sec   24    601 KBytes
[  5]   1.00-2.00   sec   416 MBytes  3.49 Gbits/sec    0    941 KBytes
[  5]   2.00-3.00   sec   453 MBytes  3.80 Gbits/sec    0   1.13 MBytes
[  5]   3.00-4.00   sec   448 MBytes  3.75 Gbits/sec   46   1.24 MBytes
[  5]   4.00-5.00   sec   429 MBytes  3.60 Gbits/sec    0   1.29 MBytes
[  5]   5.00-6.00   sec   435 MBytes  3.65 Gbits/sec    0   1.33 MBytes
[  5]   6.00-7.00   sec   451 MBytes  3.79 Gbits/sec    0   1.39 MBytes
[  5]   7.00-8.00   sec   439 MBytes  3.68 Gbits/sec    0   1.41 MBytes
[  5]   8.00-9.00   sec   441 MBytes  3.70 Gbits/sec    0   1.43 MBytes
[  5]   9.00-10.00  sec   432 MBytes  3.63 Gbits/sec    0   1.44 MBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  4.14 GBytes  3.56 Gbits/sec   70             sender
[  5]   0.00-10.00  sec  4.14 GBytes  3.55 Gbits/sec                  receiver

iperf Done.
```

(d) worker-1 client: 3.45 Gbits/sec, 1168 retransmissions

```
netlab@cloud-client:~/kubernetes-iperf3$ kubectl exec -it iperf3-clients-rkkvb -- iperf3 -c iperf3-server
Connecting to host iperf3-server, port 5201
[  5] local 10.0.1.2 port 59310 connected to 10.255.43.75 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec   437 MBytes  3.67 Gbits/sec    0   2.01 MBytes
[  5]   1.00-2.00   sec   421 MBytes  3.53 Gbits/sec    0   2.37 MBytes
[  5]   2.00-3.00   sec   415 MBytes  3.48 Gbits/sec    0   2.75 MBytes
[  5]   3.00-4.00   sec   428 MBytes  3.59 Gbits/sec    0   2.88 MBytes
[  5]   4.00-5.00   sec   394 MBytes  3.30 Gbits/sec  231   2.88 MBytes
[  5]   5.00-6.00   sec   420 MBytes  3.52 Gbits/sec  154   2.88 MBytes
[  5]   6.00-7.00   sec   389 MBytes  3.26 Gbits/sec  591   2.88 MBytes
[  5]   7.00-8.00   sec   409 MBytes  3.43 Gbits/sec   64   2.10 MBytes
[  5]   8.00-9.00   sec   399 MBytes  3.35 Gbits/sec  128   1.51 MBytes
[  5]   9.00-10.00  sec   401 MBytes  3.37 Gbits/sec    0   1.64 MBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  4.02 GBytes  3.45 Gbits/sec  1168             sender
[  5]   0.00-10.00  sec  4.01 GBytes  3.45 Gbits/sec                  receiver

iperf Done.
```

Figure 7: iperf3 output for Q10: pod placement and summarized throughput/retransmissions from the control-plane, worker-2, and worker-1 clients.